OS Analytics and Bottleneck Classifier

A Modular Telemetry-Driven Framework for Operating System Bottleneck Classification and LLM-Assisted Root-Cause Remediation

Himanshu (SAP ID: 590024811) • Saksham Mehta (SAP ID: 590025628)

Lavish Gupta (SAP ID: 590026931) • Piyush Sindhu (SAP ID: 590028080)

B.Tech. Computer Science and Engineering | University of Petroleum and Energy Studies (UPES), India

Abstract

Operating-system performance degradation often emerges from interactions among CPU scheduling, memory pressure, storage I/O and other resource domains. Conventional threshold-based monitoring can produce noisy alerts or fail to distinguish the active bottleneck when several counters change together. This paper presents a modular telemetry-driven framework that samples operating-system counters, classifies four operational states, and connects the prediction to a structured root-cause analysis (RCA) assistant. The packaged implementation includes a workload-generation pipeline, run-grouped preprocessing, a model benchmark suite, a Streamlit dashboard supporting live telemetry and CSV playback, and a Gemini-oriented diagnosis agent with an offline heuristic fallback. The included raw dataset contains 2,400 observations across 32 runs, balanced across Normal, CPU_Bound, Memory_Pressure and IO_Contention. The saved split contains 1,950 training rows from 26 runs and 450 test rows from six disjoint runs. In the packaged benchmark artifact, XGBoost, LightGBM and Random Forest each report 100% held-out test accuracy and a Macro F1 of 1.0000; XGBoost has a recorded mean inference latency of 0.0071 ms. These results are promising within the controlled dataset, but they do not establish production generalization. Further validation on independent hardware and real multi-tenant workloads is required.

Index Terms: Operating systems, performance telemetry, bottleneck classification, machine learning, XGBoost, grouped validation, root-cause analysis, AIOps, large language models.

I. INTRODUCTION

Modern computing nodes execute diverse workloads that compete for processor time, physical memory, storage bandwidth and network capacity. The resulting performance symptoms are not always isolated. For example, sustained storage writes may trigger dirty-page writeback, increase kernel activity and leave processes waiting for I/O. A monitor that considers only one counter can misinterpret this coupled behavior. Tools such as top, vmstat, sar and iostat expose valuable telemetry, yet an operator must still interpret signals across subsystems and decide what to investigate next.

The project addresses this gap with a two-tier design. A local machine-learning classifier maps numeric telemetry to a bottleneck class with low inference cost. A separate diagnosis component uses the predicted class and telemetry context to produce a structured explanation, investigation commands and remediation suggestions. Separating classification from language-model assistance keeps the fast prediction path distinct from the slower, network-dependent explanation path.

A. Problem Statement

The objective is to build a reproducible and non-intrusive pipeline that samples OS performance counters at approximately 1 Hz, classifies non-linear resource bottleneck regimes without row-level temporal leakage, and provides actionable root-cause guidance. The four target classes are Normal, CPU_Bound, Memory_Pressure and IO_Contention.

B. Contributions

The implementation combines (1) configurable workload generation and telemetry capture, (2) a run-aware holdout split and grouped cross-validation for model evaluation, (3) comparison of a static heuristic with classical ML and neural models, and (4) an interactive dashboard connected to an LLM-assisted RCA component with a deterministic local fallback.

II. RELATED WORK AND MOTIVATION

The project dossier groups existing approaches into three broad families. First, static rule systems such as Nagios and Zabbix are useful for operational monitoring but depend on thresholds that may be brittle when resource signals interact. They are discussed here as related work; the supplied project files do not establish that either product is integrated into this implementation. Second, time-series anomaly detectors such as ARIMA and Isolation Forest can identify unusual behavior, but anomaly detection alone does not necessarily assign a meaningful OS bottleneck label. Third, deep-learning approaches for system logs can require text processing and may not suit a lightweight sampling loop. LLM-based operational assistants add explanation capabilities, but their network dependency and response latency motivate a fast local classifier and an offline fallback.

The design inference is that tabular telemetry can be handled effectively by tree ensembles, while a language model is reserved for contextual diagnosis rather than the core classification decision. This is an architectural rationale, not a claim that every prior system has the same performance limitations in every deployment.

III. SYSTEM DESIGN AND METHODOLOGY

A. Telemetry and Target Classes

The generator configuration specifies a 1.0-second sampling interval, eight runs per class, and a nominal 90-second duration per run, with warm-up and cooldown samples discarded. The included raw CSV has 2,400 rows and 23 columns, including metadata, telemetry features and labels. The core signals include user and system CPU percentages, CPU I/O wait and idle percentages, normalized load average, context-switch and interrupt rates, memory use and availability, swap use and traffic, disk read/write throughput, disk busy percentage, and network receive/transmit throughput. The CSV contains 15 telemetry-like numeric counters when the swap in/out fields are counted alongside the other resource features.

Normal represents nominal resource headroom. **CPU_Bound** represents compute or scheduler saturation. **Memory_Pressure** represents RAM pressure and swap activity. **IO_Contention** represents storage saturation and processes waiting for I/O.

B. End-to-End Pipeline

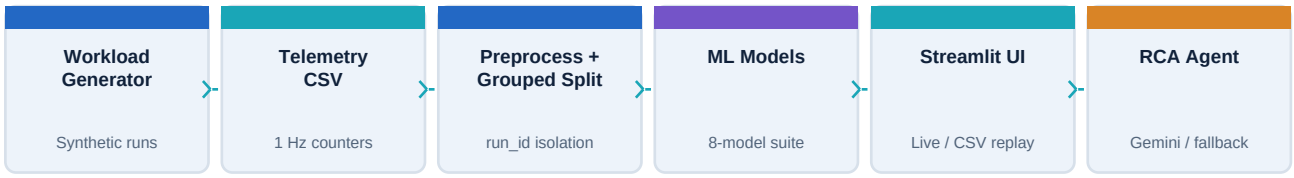

Fig. 1. High-level data and inference flow in the supplied implementation.

The dataset generator creates independent workload runs and records telemetry. Preprocessing validates required metadata, converts features to numeric values, handles infinite values as missing, removes constant features when configured, encodes labels and writes train/test CSV files. The training component benchmarks a static heuristic, Logistic Regression, Random Forest, XGBoost, LightGBM, an RBF-kernel SVM, an MLP and a 1D CNN. Trained artifacts are then loaded by the Streamlit application. The RCA agent attempts to use Gemini when credentials are configured; otherwise, a local rule-based response is available.

C. Leakage-Aware Evaluation

The split unit is run_id rather than individual rows. The saved split manifest records 26 training runs and six test runs, with no run overlap intended by the preprocessing code. The training configuration uses StratifiedGroupKFold with five folds for the classical model search. Standardization and imputation are performed in model pipelines or within the neural model preparation, rather than fitting transformations on the held-out test rows. This design reduces the risk that adjacent observations from the same workload run appear in both training and testing.

The split is not perfectly balanced by run count across labels: the packaged manifest reports six or seven training runs per class and one or two test runs per class. The total is 1,950 training observations and 450 test observations. As a result, the held-out score should be interpreted with the small number of independent test runs in mind.

D. Metrics

Accuracy is the fraction of correctly classified test observations. Macro precision, macro recall and Macro F1 average the corresponding class-level metrics so that each class contributes equally. Inference latency is reported in milliseconds per observation as stored in the model-comparison artifact. Latency values are environment-dependent and should be treated as measurements from the packaged run, not universal guarantees.

IV. IMPLEMENTATION

A. Software Components

Component	Role in the framework
generate_dataset.py	Creates workload runs and collects OS telemetry using psutil and configurable CPU, memory and I/O workloads.
preprocess.py	Validates telemetry, creates run-grouped train/test partitions, encodes labels and saves a split manifest.
train.py	Tunes/evaluates classical models and trains the 1D CNN; exports metrics and model artifacts.
app.py	Provides the Streamlit interface, model loading, live telemetry and CSV-based analysis paths.
llm_agent.py	Builds structured diagnosis reports and uses an offline heuristic fallback when the Gemini client is unavailable.

B. Diagnosis Output and Operational Safety

The diagnosis schema includes severity, root cause, technical analysis, investigation commands and remediation commands. A schema-validated report improves consistency of the output shape, but does not guarantee that every suggested command is safe for every machine. Remediation should therefore be reviewed by an operator before execution, especially commands that change kernel settings, process priority or cache state. The offline fallback improves availability but uses predefined class-oriented explanations rather than a live, context-rich language-model response.

V. EXPERIMENTAL RESULTS

Table I reproduces the metrics stored in *artifacts/model_comparison.csv* inside the supplied ZIP. The table is based on the packaged benchmark artifact and was not regenerated during preparation of this paper.

TABLE I. PACKAGED MODEL BENCHMARK RESULTS

Model	CV Macro F1	Accuracy	Precision	Recall	Test Macro F1	Latency (ms)
Static Rule Baseline	N/A	0.5511	0.5677	0.6633	0.5541	0.0843
Logistic Regression	0.9653	0.9978	0.9967	0.9983	0.9975	0.0044
Random Forest	0.9993	1.0000	1.0000	1.0000	1.0000	0.1481
XGBoost	0.9935	1.0000	1.0000	1.0000	1.0000	0.0071
LightGBM	0.9988	1.0000	1.0000	1.0000	1.0000	0.0145
SVM RBF	0.9556	0.9956	0.9951	0.9950	0.9950	0.0150
MLP Deep Net	0.9613	0.9978	0.9967	0.9983	0.9975	0.0046
1D CNN Net (DL)	N/A	0.9844	0.9777	0.9850	0.9810	0.0059

A. Results Interpretation

The packaged artifact reports 100% test accuracy and Macro F1 for Random Forest, XGBoost and LightGBM. XGBoost has a recorded latency of 0.0071 ms per observation in the CSV artifact; LightGBM records 0.0145 ms and Random Forest 0.1481 ms. Logistic Regression and the MLP report 99.78% accuracy, while the SVM reports 99.56%. The 1D CNN reports 98.44% accuracy. The static heuristic baseline reports 55.11% accuracy and a Macro F1 of 0.5541, indicating that the configured thresholds perform substantially worse than the trained models on this test split.

These scores should not be interpreted as evidence of perfect real-world classification. The dataset is synthetic and deliberately generated under controlled workload classes. The test set contains only six independent runs. Real machines may exhibit mixed workloads, hardware differences, background services, changing kernel versions and ambiguous transitions between classes. Additional external testing is needed before operational claims can be made.

VI. DISCUSSION

The results support the project's design choice to keep numeric classification local and use the language model for explanation. The classical tree ensembles are competitive on the controlled tabular dataset, while the 1D CNN does not show a benefit in the reported test metrics. The static heuristic illustrates how fixed rules can underperform when the workload signatures are more complex than the small set of manually selected thresholds.

There are important validity limits. First, the sample population is a controlled synthetic dataset rather than a collection of independent production incidents. Second, six held-out runs provide limited evidence about variation across environments. Third, inference latency depends on hardware, software versions, batch shape and measurement method. Fourth, LLM output quality, API availability and remediation safety require separate evaluation from classifier accuracy. Finally, the reported near-perfect test results should be replicated with newly generated runs and, ideally, real telemetry from distinct hosts.

VII. CONCLUSION AND FUTURE WORK

This paper described a modular framework for classifying operating-system bottlenecks from telemetry and connecting predictions to structured root-cause guidance. The supplied implementation includes workload generation, run-grouped preprocessing, an eight-model benchmark suite, saved model artifacts, a Streamlit dashboard and a Gemini-oriented RCA component with an offline fallback. The packaged benchmark reports strong results for tree-based models on the controlled held-out split, while the static-rule baseline performs substantially worse.

Future work should validate the classifier on real multi-tenant workloads and different hardware, report confidence intervals across repeated run-grouped splits, measure end-to-end dashboard and RCA latency, evaluate diagnosis correctness and command safety, explore eBPF-based telemetry, and extend the class set to additional conditions such as network socket starvation and thread deadlock states.

ACKNOWLEDGMENT

This paper was prepared from the supplied project dossier and the packaged OS Bottleneck Classifier code, datasets and benchmark artifact. Mentor details were not supplied and are therefore not listed.

REFERENCES

[1] B. Gregg, *Systems Performance: Enterprise and the Cloud*, 2nd ed. Addison-Wesley, 2020.

[2] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, 2016.

[3] G. Ke et al., “LightGBM: A Highly Efficient Gradient Boosting Decision Tree,” in *Advances in Neural Information Processing Systems*, 2017.

[4] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[5] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[6] Google DeepMind, “Gemini: A Family of Highly Capable Multimodal Models,” 2024. Project dossier reference; bibliographic details should be verified before formal submission.